

Smart Farmer Procurement and Queue Management System (AgriQueue)

A

Project-I Report

Submitted

In Partial Fulfilment
For the award of the Degree of
Bachelor of Technology
(VII Semester)



Supervised By:

Mrs. Anju Jangid
HOD

Submitted By:

[Student Name]
Roll No: [Roll No]

Department of Computer Science and Engineering
Jodhpur Institute of Engineering and Technology(Autonomous)
JIET Universe, Jodhpur
Session 2026-27

Certificate

This is to certify that the project entitled “**Smart Farmer Procurement and Queue Management System (AgriQueue)**” has been carried out by the students of **Jodhpur Institute of Engineering & Technology, Jodhpur** under my guidance and supervision in partial fulfillment of the degree of Bachelor of Technology in Computer Science and Engineering of Bikaner Technical University during the academic year of 2026-27.

The project work is original and has not been submitted elsewhere for the award of any degree or diploma.

Date:

Place: Jodhpur

Anju Jangid

HOD

Acknowledgment

We express our sincere gratitude to all those who have been instrumental in the completion of this project report on “*Smart Farmer Procurement and Queue Management System (AgriQueue)*”, which represents a significant step toward digital transformation in Indian agricultural procurement.

We are deeply thankful to our project guide **Mrs. Anju Jangid**, *HOD*, Department of Computer Science and Engineering, JIET, Jodhpur, for the invaluable guidance, continuous encouragement, and insightful feedback throughout the project development process. Their expertise in software engineering and web technologies has been the cornerstone of this project.

We extend special gratitude to the **Head of Department**, Department of Computer Science and Engineering, for providing all necessary facilities and an encouraging academic environment that enabled us to work on this innovative project.

We are thankful to the **Jodhpur Institute of Engineering and Technology (Autonomous)**, JIET Universe, Jodhpur, for providing the infrastructure, resources, and the academic framework under which this project was conceived, developed, and tested.

We also extend our appreciation to all the **faculty members** of the CSE Department who offered technical guidance and constructive criticism at various stages of the project.

Lastly, we express our gratitude to our **family members and friends** whose moral support and motivation helped us persevere through the challenges encountered during the development of this system.

[Student Name(s)]

Roll No: [Roll No]

Date: September 25, 2026

Contents

LIST OF FIGURES	iv
LIST OF TABLES	v
1 Introduction	1
1.1 Background of the Study	1
1.2 Problem Statement	1
1.3 Objectives	1
1.4 Overall System Architecture and Concept	2
1.5 Scope	2
1.6 Significance	3
1.7 Methodology Overview	3
2 Literature Survey	4
2.1 Introduction	4
2.2 Digital Agricultural Procurement Systems	4
2.3 Online Slot Booking for Agricultural Procurement	4
2.4 Queue Management in Agricultural Procurement	4
2.5 Real-Time Notification and Procurement Tracking	5
2.6 Artificial Intelligence and AI Agents in Procurement	5
2.7 Existing Work in the Field	5
2.8 Research Gap Identified	5
2.9 Research Papers Referred	5
3 Requirement Specification & Methodology	6
3.1 Software Development Life Cycle (SDLC) Model	6
3.2 System Use Cases	6
3.3 Functional Requirements	7
3.3.1 Farmer Module	7
3.3.2 Staff Module	7
3.3.3 Admin Module	8

3.4	Non-Functional Requirements	8
3.5	System Interfaces	8
3.5.1	Software Interfaces	8
3.5.2	Communication Interfaces	9
3.6	Hardware and Operating Specifications	9
3.7	General Constraints	9
4	Work Distribution	10
4.1	Team Task Allocation	10
4.2	Module Descriptions	11
4.3	Tools Used	11
5	Design Document	12
5.1	System Architecture	12
5.2	System Use Cases	14
5.3	Data Flow Diagrams (DFD)	14
5.3.1	DFD Level 0: Context-Level Diagram	14
5.3.2	DFD Level 1: Subsystem Data Flow Decomposition	16
5.4	Entity Relationship Model	17
5.5	Database Schema Design	18
5.6	Farmer Booking Process Flow	18
5.7	Staff Procurement Verification Flow	20
5.8	System Interaction Sequence Diagrams	21
5.8.1	Slot Booking and Predictive Wait-Time Allocation Sequence	21
5.8.2	Physical Procurement Weighment and Payment Settlement Sequence	21
5.8.3	Real-Time Live Queue SSE Synchronization Sequence	22
5.9	Cloud Deployment Topology	23
5.10	System Functional Modules	23
6	Experimental Setup	25
6.1	Development Environment	25
6.2	Installation Process	25
6.2.1	Backend Setup	25
6.2.2	Frontend Setup	26
6.3	Execution Commands	26
6.4	ML Model Training Details	26
6.5	Hardware Configuration	27
6.6	API Endpoints Summary	27

7	Test Plan Document	29
7.1	Test Strategy	29
7.2	Test Cases and Status Report	29
7.3	ML Model Metrics	31
7.4	Performance Results	31
7.5	Security Test Results	32
8	Results	33
8.1	Implementation Achievements	33
8.2	Screenshots of Working Modules	33
8.3	ML Prediction Sample Results	37
8.4	Outputs and Observations	37
9	Conclusion & Future Work	38
9.1	Conclusion	38
9.2	Limitations	38
9.3	Future Work	39
10	Glossary	40
11	References	43

List of Figures

1.1	AgriQueue Comprehensive Architecture, Module Interfaces, and Operational Workflow	2
3.1	Agile Iterative Sprint Lifecycle for AgriQueue	6
3.2	Comprehensive UML Use Case Diagram for AgriQueue Actors and System Boundary	7
5.1	AgriQueue Three-Tier Modular System Architecture	13
5.2	DFD Level 0: AgriQueue Context-Level Data Flow Diagram	15
5.3	DFD Level 1: Subsystem Process Decomposition and Data Stores	16
5.4	AgriQueue Relational Entity Relationship Diagram	17
5.5	Farmer Slot Booking and Queue Progression Flowchart	19
5.6	Staff Procurement Physical Verification and Billing Flow	20
5.7	End-to-End Slot Booking and ML Wait-Time Prediction Sequence Diagram	21
5.8	Physical Procurement Weighment and Digital Payment Settlement Sequence Diagram	22
5.9	Server-Sent Events (SSE) Live Queue Sequence Diagram	22
5.10	Production Cloud Deployment Topology	23
8.1	AgriQueue Public Landing Page and Discovery Portal	34
8.2	Farmer Dashboard with KPI Summary and AI Multilingual Assistant	34
8.3	Slot Booking Workflow with Interactive Calendar and Slot Capacity	35
8.4	Booking Confirmation with Token Issuance (C-048) and AI Wait Estimate .	35
8.5	Farmer Live Queue Real-Time Tracking Interface	36
8.6	Staff Queue Management Console with Active Counter Workflow	36

List of Tables

3.1	AgriQueue Non-Functional Requirements Matrix	8
3.2	System Hardware Requirements	9
4.1	Work Distribution among Team Members	10
5.1	Detailed System Use Cases by Role	14
5.2	Primary Table Schemas	18
6.1	Development Environment	25
6.2	Execution Commands	26
6.3	Hardware Configuration	27
6.4	Key API Endpoints	28
7.1	Test Case Status Report	29
7.2	ML Model Validation Results	32
8.1	ML Wait Time Prediction Samples	37
10.1	Glossary of Terms and Abbreviations	40

Chapter 1

Introduction

1.1 Background of the Study

- Indian agriculture supports 600+ million farmers and contributes ~17% to national GDP.
- Government MSP procurement operates through thousands of procurement centers (mandis) across India.
- Farmers face severe inefficiencies at procurement centers:
 - Uncontrolled physical queues stretching for kilometers.
 - No visibility into wait times or slot availability before arrival.
 - Manual procurement recording – slow, error-prone, unaudited.
 - Weeks of delay in payment with no tracking visibility.
 - No digital communication channel between farmer and center.

1.2 Problem Statement

- No pre-booking system – all farmers arrive simultaneously at peak hours.
- Physical queue is invisible – farmers cannot see their position or wait time.
- Farmer, staff, and admin operate in information silos – no unified platform.
- Post-procurement payment lifecycle is opaque and manually managed.
- No data-driven approach for predicting or balancing queue wait times.

1.3 Objectives

- Build a full-stack, role-based web platform for Farmer, Staff, and Admin.
- Implement slot booking with capacity enforcement for procurement centers.
- Provide real-time live queue via Server-Sent Events (SSE) streaming.
- Integrate an ML-based wait time predictor (HistGradientBoosting, $R^2 = 0.988$).
- Enable digital procurement verification (weight, quality, rate, net payable).

- Build a complete payment lifecycle with settlement and dispute management.
- Deliver Web Push Notifications for real-time farmer alerts.
- Integrate an AI command agent (LangChain + Google Gemini) for NL queries.

1.4 Overall System Architecture and Concept

Figure 1.1 provides a comprehensive architectural and operational overview of the AgriQueue ecosystem.

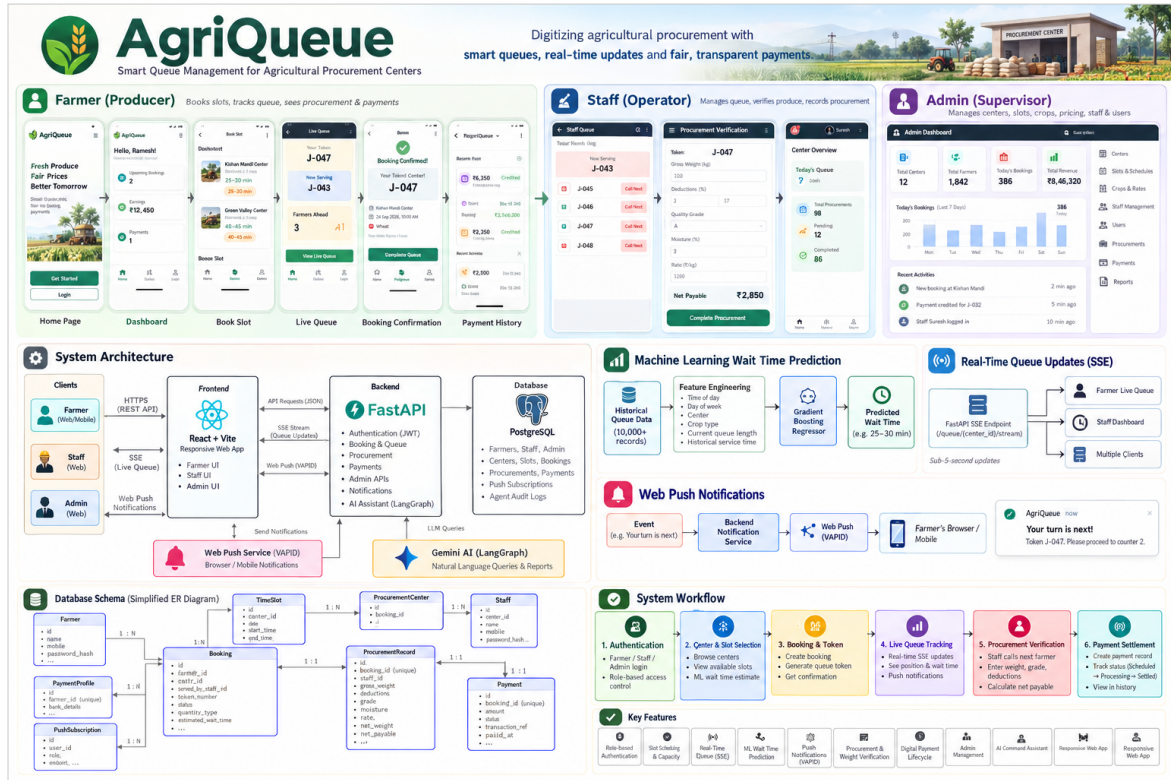


Figure 1.1: AgriQueue Comprehensive Architecture, Module Interfaces, and Operational Workflow

The platform comprises three primary functional pillars:

- **Farmer Module:** Advance slot reservation, digital token generation, live SSE queue monitoring, and transparent payment settlement tracking.
- **Staff Module:** Center-specific queue dispatching, automated weighment entry, moisture deductions, and instant MSP amount computation.
- **Admin Module:** Centralized management of mandis, capacity limits, crop MSP schedules, staff allocations, and dispute resolution.

1.5 Scope

- **User roles:** Farmer, Staff (center operator), Admin (government/agency).
- **Produces:** Wheat, Paddy, Cotton, Mustard, Maize, Soybean, Chana, Sugarcane.

- **Queue:** Token-based SSE streaming with 5-second refresh interval.
- **Deployment:** Cloud backend (FastAPI + PostgreSQL), Vercel frontend (React).
- **Out of scope:** Native mobile app, hardware weighbridge integration, SMS gateway.

1.6 Significance

- Farmers save fuel cost and time by planning visits with advance slot booking.
- Procurement centers increase throughput via digital capacity management.
- Digital audit trail eliminates manual errors and reduces corruption.
- ML analytics enable evidence-based capacity planning for administrators.
- Aligns with Government of India's **Digital India** and **PM-KISAN** initiatives.

1.7 Methodology Overview

1. Requirement gathering and stakeholder analysis.
2. System architecture and database schema design.
3. Backend API development (FastAPI, SQLAlchemy, asyncpg).
4. ML model training (HistGradientBoosting on synthetic queue data).
5. Frontend development (React 19, Vite, custom CSS).
6. Integration testing – full farmer-to-payment workflow.
7. Cloud deployment and production validation.

Chapter 2

Literature Survey

2.1 Introduction

- **Review Scope:** Covers digital procurement, slot booking, virtual queues, real-time alerts, ML wait-times, and conversational AI agents.
- **Core Objective:** Identifies traditional mandi bottlenecks to establish the technical foundation for AgriQueue.

2.2 Digital Agricultural Procurement Systems

- **State of the Art:** Portals like e-NAM digitize trade records, farmer registry, and price discovery.
- **Operational Gap:** Existing portals lack physical crowd scheduling, arrival quotas, and yard queue control.

2.3 Online Slot Booking for Agricultural Procurement

- **Reservation Paradigm:** Adapts appointment models from healthcare and banking to mandi operations.
- **Traffic Regulation:** Distributes arrivals evenly across operational hours, eliminating morning congestion.

2.4 Queue Management in Agricultural Procurement

- **Conventional Bottleneck:** Unregulated queues cause prolonged waits, produce spoilage, and farmer fatigue.
- **Virtual Queuing:** Implements sequential token generation, real-time position tracking, and counter calling.

2.5 Real-Time Notification and Procurement Tracking

- **Event Streaming & Alerts:** Employs Server-Sent Events (SSE) for live queues and Web Push for proactive alerts across booking, turn-calling, weighment, and payments.

2.6 Artificial Intelligence and AI Agents in Procurement

- **ML Wait-Time Estimation:** Applies gradient boosting on queue depth and history to predict wait times.
- **Conversational AI Agents:** Uses LLM ReAct agents (LangChain/Gemini) for farmer queries and assistance.

2.7 Existing Work in the Field

1. **Digital Mandi Portals:** Streamline pricing, but lack arrival scheduling.
2. **Online Booking Platforms:** Schedule slots, but lack weighbridge/MSP workflows.
3. **Digital Queue Systems:** Standardize token calling, but unsuited for bulk intake.
4. **AI Delay & Chat Tools:** Forecast delays or FAQs, but lack mandi workflow context.

2.8 Research Gap Identified

- **Fragmented Architecture:** Registration, pricing, tokens, and payments operate in disjointed silos.
- **Zero Pre-Arrival Visibility:** Farmers travel to mandis without knowing queue backlog or wait times.
- **AgriQueue Solution:** Integrates slots, SSE queues, ML, WebPush, and a Gemini AI agent.

2.9 Research Papers Referred

1. **Gumbi et al. (2023):** Smallholder digital adoption barriers; guides conversational AI agent.
2. **Swain et al. (2022):** Evaluated mandi arrivals under e-NAM; demonstrated need for queue coordination.
3. **Han et al. (2024):** ML in appointment scheduling; informs capacity-enforced slot allocation.
4. **Pak et al. (2021):** Regression models for queue delays; guides wait estimation.

Chapter 3

Requirement Specification & Methodology

3.1 Software Development Life Cycle (SDLC) Model

AgriQueue adopted an Agile Iterative methodology tailored for rapid multi-tier development, iterative machine learning model tuning, and continuous stakeholder feedback:

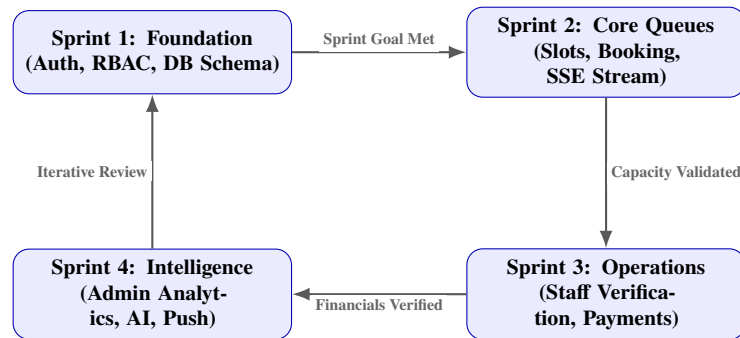


Figure 3.1: Agile Iterative Sprint Lifecycle for AgriQueue

- **Sprint 1 (Weeks 1–2):** Domain modeling, database migrations, JWT authentication, and center/crop configuration.
- **Sprint 2 (Weeks 3–4):** Slot capacity logic, booking engine, token assignment, and live queue SSE streaming.
- **Sprint 3 (Weeks 5–6):** Staff operational workflow, weighment deductions, automated net calculation, and payment lifecycle.
- **Sprint 4 (Weeks 7–8):** HistGBR model training, LangGraph AI command agent, WebPush alerts, and cloud deployment.

3.2 System Use Cases

The platform supports distinct actor interactions across the three operational roles:

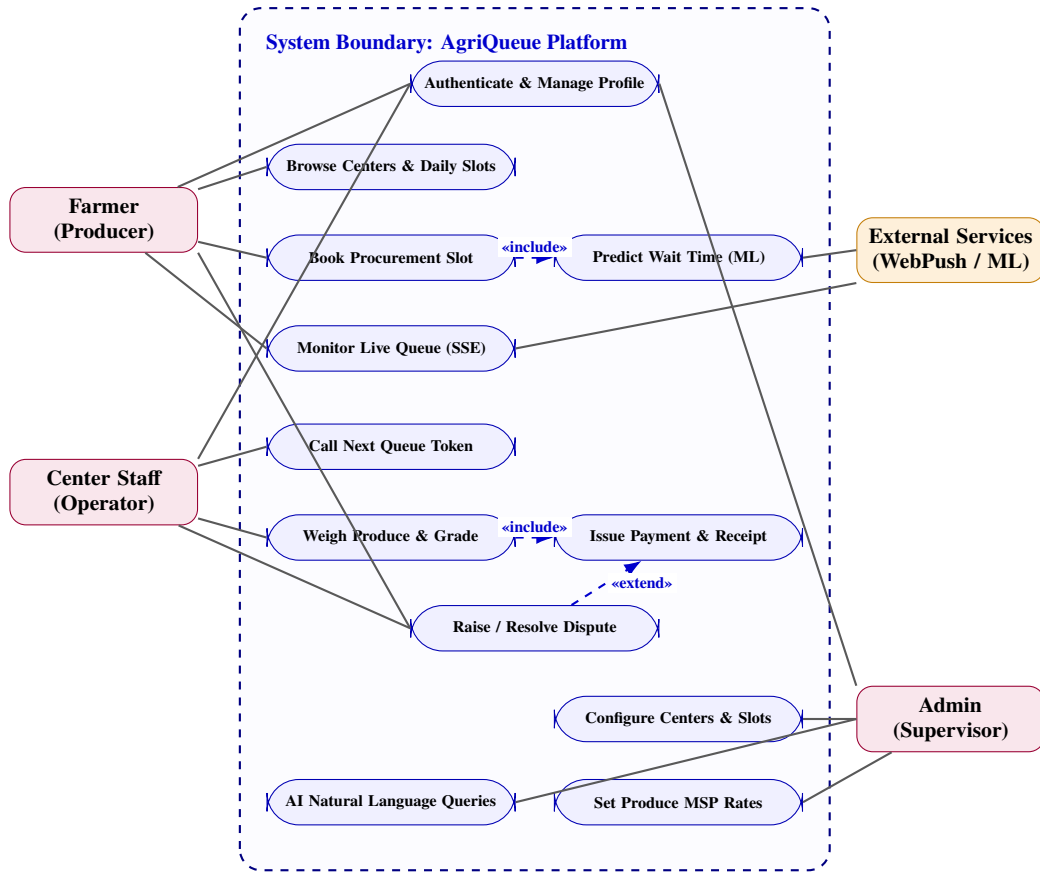


Figure 3.2: Comprehensive UML Use Case Diagram for AgriQueue Actors and System Boundary

3.3 Functional Requirements

3.3.1 Farmer Module

- Self-registration with validated mobile number, full name, district, and village.
- Secure login with HTTP-only cookie-based authentication.
- Real-time discovery of nearby procurement centers and operational crop types.
- Inspection of daily time slots with visual availability indicators (Available, Full, Expired).
- Slot reservation with dynamic capacity validation and ML-estimated wait calculation.
- Live token tracking via low-latency Server-Sent Events with countdown metrics.
- Inspection of procurement receipts and tracking payment settlement states.
- Browser push notification subscription for queue summons and disbursement alerts.

3.3.2 Staff Module

- Role-authenticated login restricted to designated procurement center locations.
- Real-time dashboard of daily registered arrivals sorted by sequential token numbers.
- Immediate status mutation to *Serving* with synchronized client-side notification.

- Digital entry of gross vehicle weight, tare weight, tare deductions, and moisture percentage.
- Automated evaluation of net payable amount using government MSP lookup tables.
- Initiation and status management of digital payment transactions (Scheduled → Settled).

3.3.3 Admin Module

- System-wide operational dashboard visualizing total centers, bookings, and financial turnover.
- Administrative CRUD controls for centers, operating hours, capacity quotas, and crop MSP rates.
- Staff profile provisioning and center assignment management.
- Audit trail logging and administrative review of user dispute tickets.

3.4 Non-Functional Requirements

Table 3.1: AgriQueue Non-Functional Requirements Matrix

Category	Specification and Threshold
Performance	95% of REST API responses served in under 500 milliseconds
Real-Time Delivery	Queue mutations broadcast via SSE to connected clients within 5 seconds
Concurrency	Architectural support for 1,000+ simultaneous active farmer sessions
Security	Argon2id password hashing, HTTP-only JWT storage, and CORS whitelisting
ML Precision	Wait-time prediction error within ± 10 minutes ($R^2 \geq 0.95$)
Availability	99.5% system uptime maintained throughout operational hours (8 AM – 6 PM)

3.5 System Interfaces

3.5.1 Software Interfaces

- **User Interface:** React 19 single-page application utilizing Lucide icons and modern CSS.
- **Application Framework:** FastAPI ASGI web framework running under Uvicorn.
- **Relational Database:** PostgreSQL 15 accessed asynchronously through SQLAlchemy 2.0 and asyncpg.
- **Inference Engine:** scikit-learn HistGradientBoosting regression pipeline serialized via Joblib.

- **Generative Agent:** LangChain and LangGraph orchestration over Google Gemini Flash APIs.
- **Push Protocols:** WebPush VAPID protocol managed through pywebpush.

3.5.2 Communication Interfaces

- **HTTPS REST API:** Standardized JSON payloads over TLS 1.3 encryption.
- **Server-Sent Events (SSE):** Persistent HTTP text/event-stream connection for unidimensional push.
- **Browser WebPush:** Secure VAPID-authenticated event delivery across disparate networks.

3.6 Hardware and Operating Specifications

Table 3.2: System Hardware Requirements

Resource	Minimum Specification	Recommended Production
Server CPU	2 Cores (x86-64 / ARM64)	4+ Cores (2.5+ GHz)
Server RAM	4 GB	8–16 GB High-Speed DDR4
Disk Storage	20 GB NVMe SSD	100+ GB Managed Cloud Storage
Network	10 Mbps Dedicated Uplink	100+ Mbps Redundant Connection
Client Device	Modern Smartphone / PC	4G/5G Connectivity, 2+ GB RAM

3.7 General Constraints

- **Network Connectivity:** Active internet connection required; offline synchronization not supported in initial release.
- **Browser Compatibility:** Web Push features require modern standards-compliant browsers (Google Chrome, Mozilla Firefox, Microsoft Edge).
- **Hardware Instrumentation:** Weighbridge readings are entered manually by verified center staff rather than via IoT telemetry.
- **Localization:** User interface localization currently provided in English, with multi-language regional support scheduled for Phase II.

Chapter 4

Work Distribution

4.1 Team Task Allocation

The AgriQueue platform was engineered by a two-member development team. Work responsibilities were partitioned systematically across system architecture layers: Member A led backend architecture, FastAPI service endpoints, authentication, and the LangChain AI Command Agent; Member B led frontend client portals, user interface design, and Machine Learning model training; and database schema engineering, requirement specification, and end-to-end integration were collaboratively conducted by both members. Table 4.1 details the module-wise schedule and task assignments.

Table 4.1: Work Distribution among Team Members

Module	Start	End	Responsible
Requirement Analysis & Design	01-07-26	15-07-26	Member A, Member B
Database Schema & ERD	10-07-26	20-07-26	Member A, Member B
Authentication & User Management	16-07-26	28-07-26	Member A
Centers & Slot APIs	20-07-26	05-08-26	Member A
Booking Engine & Queue (SSE)	01-08-26	20-08-26	Member A
ML Model Training & Prediction	15-08-26	05-09-26	Member B
Staff Portal Backend (Procurement)	20-08-26	10-09-26	Member A
Admin Portal Backend	01-09-26	15-09-26	Member A
Farmer Frontend (Dashboard, Slots, Queue)	01-08-26	20-09-26	Member B
Staff & Admin Frontend	15-08-26	25-09-26	Member B
Push Notifications & AI Agent	10-09-26	22-09-26	Member A
Integration, Testing & Deployment	20-09-26	30-09-26	Member A, Member B

4.2 Module Descriptions

- **Database & ERD:** Designed 12 tables – farmers, procurement_centers, slots, bookings, procurement_records, payments, payment_profiles, staff, admins, produce, push_subscriptions, agent_audit.
- **Authentication:** JWT HttpOnly cookie login for 3 roles. Argon2id password hashing. Endpoints: login, register, /me, logout.
- **Centers & Slots:** Center listing with search/filter. Slot availability with expiry detection. Capacity computation per slot per date.
- **Booking Engine:** Booking creation with slot capacity validation, token assignment, ML wait time call, and cancellation logic.
- **Queue (SSE):** SSE generator with 5-second fresh `AsyncSessionLocal` tick. Computes farmer position, tokens ahead, real-time wait estimate.
- **ML Model:** 10,000-record synthetic dataset. HistGradientBoosting with 13 features. $R^2 = 0.988$. Serialized via joblib.
- **Staff Portal:** Token calling, procurement record entry, payment lifecycle (Scheduled → Settled), dispute management. Largest module: `staff.py` (1012 lines).
- **Admin Portal:** Full CRUD for centers, slots, produce, staff, users. System-wide analytics.
- **Push + AI Agent:** VAPID WebPush for booking/payment events. LangChain + LangGraph ReAct agent with Gemini for NL queries.

4.3 Tools Used

- **Version Control:** Git + GitHub (feature branch workflow).
- **API Testing:** Postman for manual endpoint testing.
- **Development:** VS Code with Python + ESLint extensions.
- **Deployment:** Vercel CLI (frontend), cloud SSH (backend).

Chapter 5

Design Document

5.1 System Architecture

AgriQueue is architected as a modular, three-tier cloud platform designed for scalability, low latency, and fault tolerance:

- **Tier 1 – Presentation Layer:** React 19 Single Page Application (SPA) deployed across globally distributed edge nodes on Vercel CDN. Supports responsive views tailored for farmers (mobile-first), staff (operational dashboard), and administrators (analytics portal).
- **Tier 2 – Application Layer:** FastAPI asynchronous ASGI web server running on Uvicorn workers. Houses authentication guards, the slot capacity engine, the real-time SSE queue broadcaster, the scikit-learn ML inference pipeline, and the LangGraph-powered AI assistant.
- **Tier 3 – Persistence & External Services:** Managed PostgreSQL 15 database accessed via SQLAlchemy 2.0 async ORM and the high-throughput `asyncpg` driver, alongside VAPID Web Push servers and Google Gemini LLM cloud infrastructure.

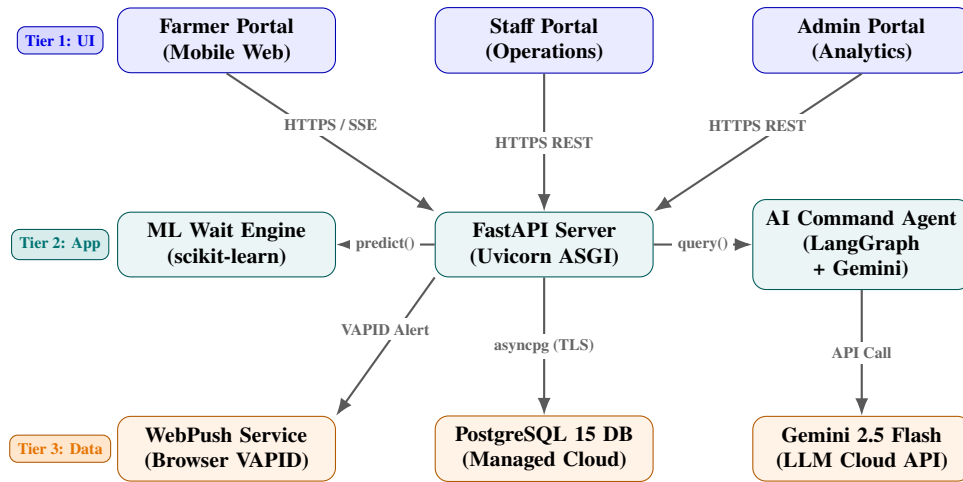


Figure 5.1: AgriQueue Three-Tier Modular System Architecture

- **Presentation Tier:** Delivers single-page responsive user interfaces rendered entirely on client devices with client-side routing via React Router.
- **Application Tier:** Stateless RESTful API server handling request validation (Pydantic), token parsing (JWT), queue sequencing, and ML wait time estimation.
- **Data Tier:** ACID-compliant relational data store with indexed queries for sub-10ms response times on active queues and booking verification.

5.2 System Use Cases

Table 5.1: Detailed System Use Cases by Role

Role	Category	Key Use Cases
Farmer	Account	Register account, Login via JWT cookie, View profile, Logout
	Booking	Search centers, View slot capacity, Book slot, Cancel booking
	Live Queue	Stream live token queue via SSE, Track estimated wait time
	Finance	View procurement receipt, Track payment status, Raise dispute
Staff	Queue Mgmt	View daily center queue, Call next token, Mark farmer serving
	Procurement	Weigh produce, Enter moisture & deductions, Compute payable
	Payments	Complete procurement, Initiate payment, Resolve farmer disputes
Admin	Center Mgmt	Add/edit procurement centers, Configure time slots & capacity
	Pricing	Set government MSP rates per produce and quality grade
	Staff & Audit	Register staff accounts, Assign centers, Monitor system analytics

5.3 Data Flow Diagrams (DFD)

Data Flow Diagrams model the flow of data through the AgriQueue platform, detailing transformations across system processes, operational actors, and persistent datastores.

5.3.1 DFD Level 0: Context-Level Diagram

The Context Diagram defines the external boundary of the platform, illustrating key inputs and outputs between AgriQueue and the primary external entities:

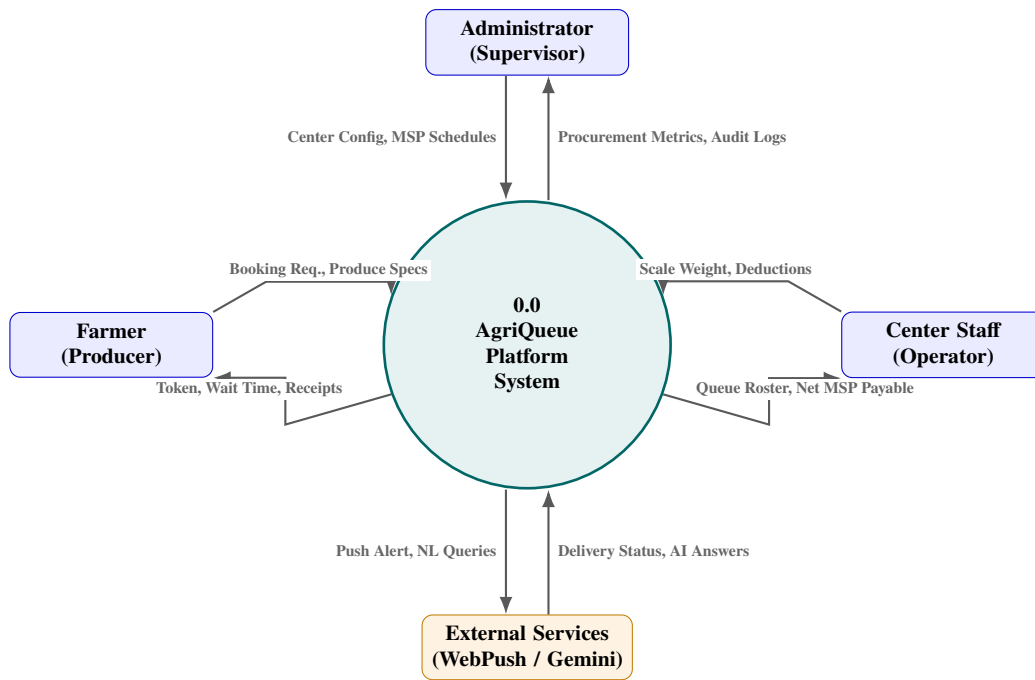


Figure 5.2: DFD Level 0: AgriQueue Context-Level Data Flow Diagram

5.3.2 DFD Level 1: Subsystem Data Flow Decomposition

The Level 1 DFD decomposes the central system into seven functional processes and five persistent datastores:

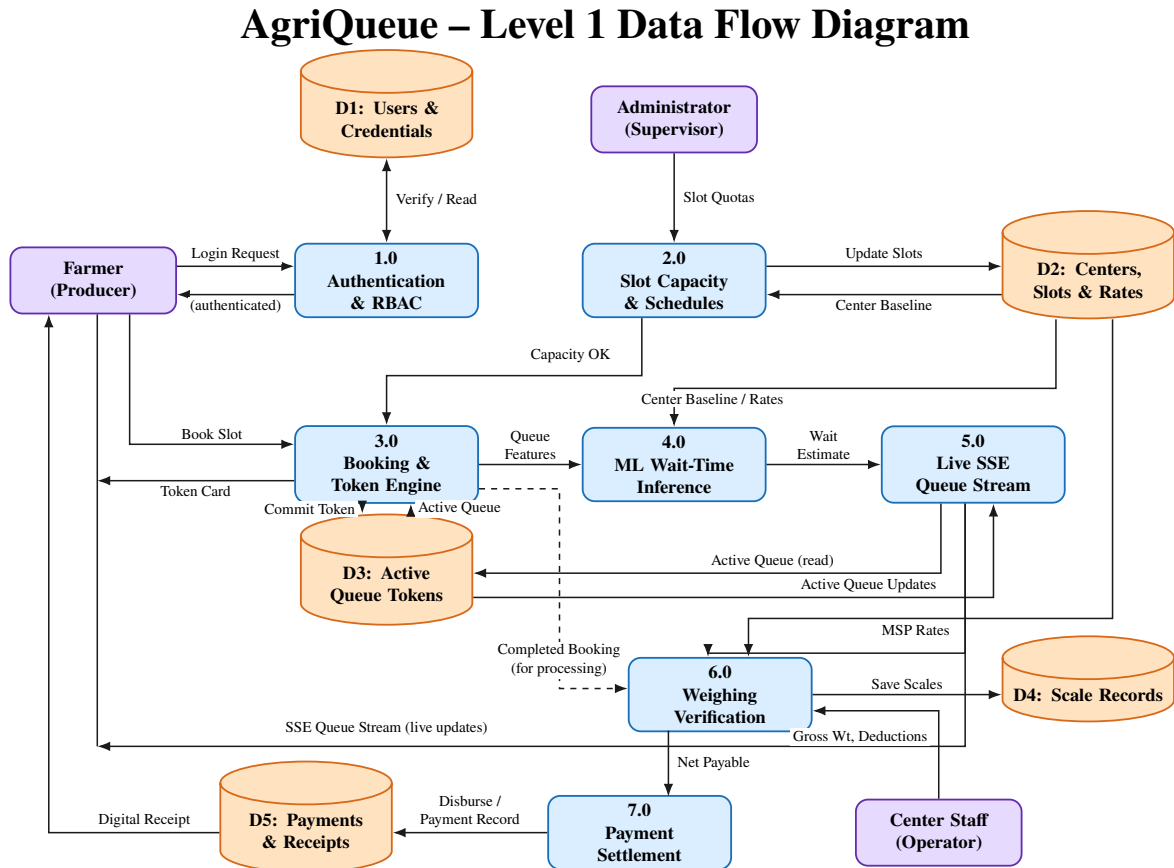


Figure 5.3: DFD Level 1: Subsystem Process Decomposition and Data Stores

5.4 Entity Relationship Model

The relational data model comprises 12 entities structured to maintain referential integrity across the booking and payment lifecycle:

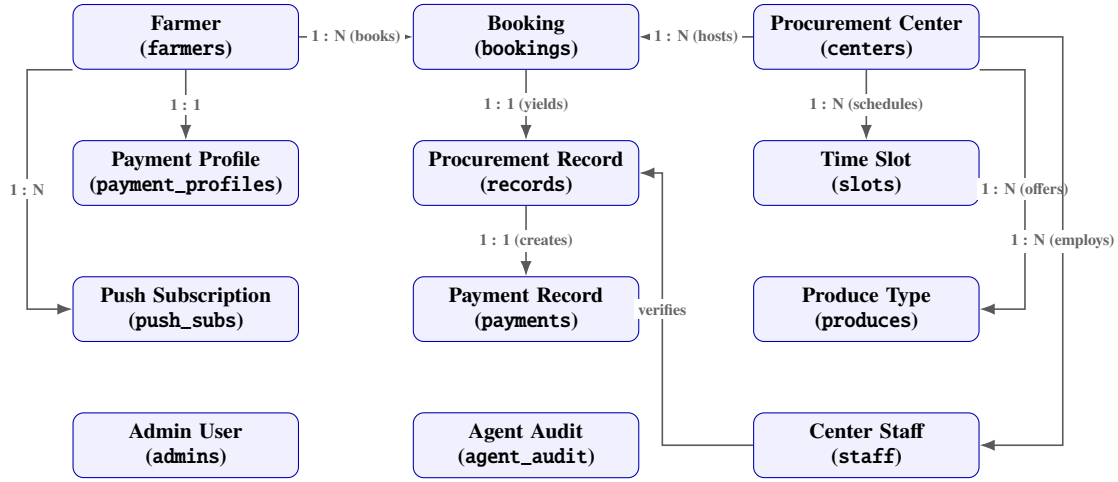


Figure 5.4: AgriQueue Relational Entity Relationship Diagram

- **Farmer to Booking (1:N):** A registered farmer can schedule multiple procurement bookings across different dates and seasons.
- **Center to Slot and Booking (1:N):** Centers partition daily operational hours into distinct slots, each enforcing an independent capacity ceiling.
- **Booking to Procurement Record (1:1):** Each completed booking produces exactly one verified physical weighing record signed off by center staff.
- **Procurement Record to Payment (1:1):** Completing a procurement automatically generates a trackable payment transaction record with automated net calculation.

5.5 Database Schema Design

Table 5.2: Primary Table Schemas

Field	Type	Constraint	Description
Table: bookings			
id	INTEGER	PK, Auto	Unique booking identifier
farmer_id	INTEGER	FK, NOT NULL	References farmers(id)
center_id	INTEGER	FK, NOT NULL	References centers(id)
slot_id	INTEGER	NOT NULL	References slots(id)
status	VARCHAR(32)	NOT NULL	Confirmed / Serving / Completed
produce	VARCHAR(64)	NOT NULL	Crop name (e.g., Wheat, Mustard)
quantity_kg	INTEGER	NOT NULL	Declared crop weight in kilograms
token_number	INTEGER	NOT NULL	Daily sequential queue token
est_wait_time	INTEGER	NOT NULL	ML predicted wait duration (minutes)
booked_at	TIMESTAMP	NOT NULL	Booking creation timestamp
Table: procurement_records			
id	INTEGER	PK, Auto	Record identifier
booking_id	INTEGER	FK, UNIQUE	References bookings(id)
actual_weight_kg	INTEGER	NOT NULL	Gross scale weight in kg
deductions_kg	INTEGER	DEFAULT 0	Moisture and impurity deductions
net_weight_kg	INTEGER	NOT NULL	Computed: actual – deductions
quality_grade	VARCHAR(8)	NOT NULL	Quality classification: Grade A / B / C
moisture_pct	FLOAT	DEFAULT 0	Measured grain moisture percentage
rate_per_kg	INTEGER	NOT NULL	Government MSP rate (Rs./kg)
net_payable	INTEGER	NOT NULL	Total disbursement: net_weight × rate
staff_id	INTEGER	FK, NOT NULL	Verifying staff operator

5.6 Farmer Booking Process Flow

The farmer booking workflow guides the agricultural producer from authentication to real-time queue participation:

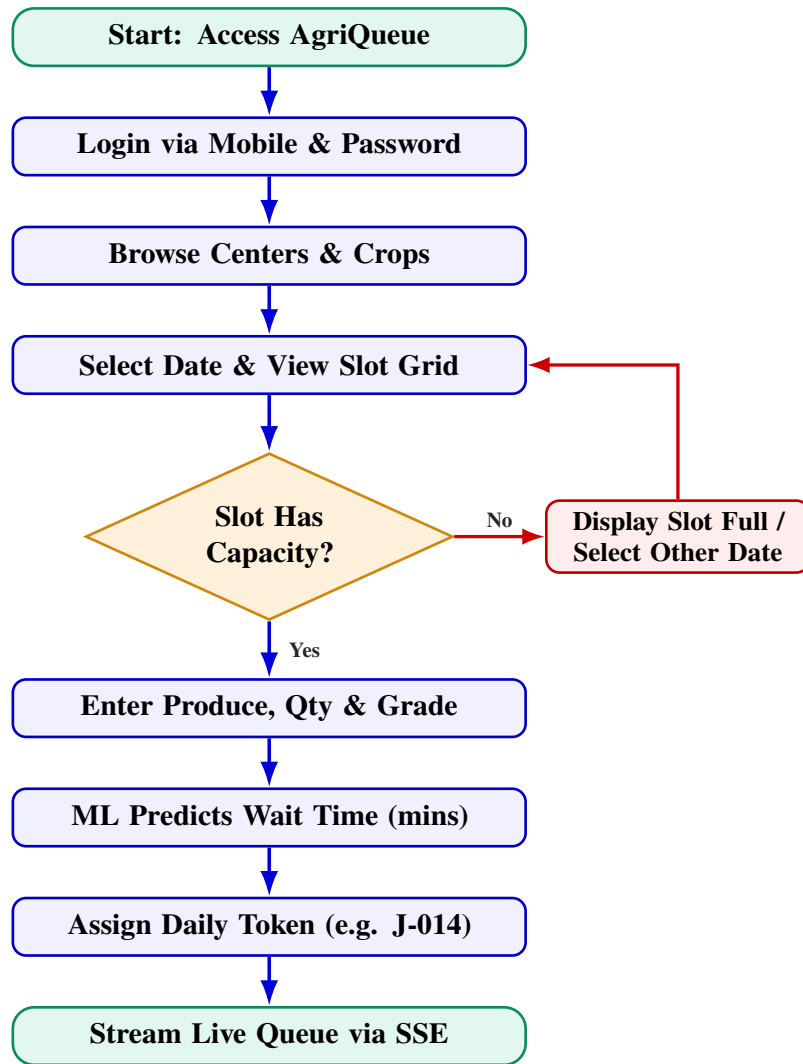


Figure 5.5: Farmer Slot Booking and Queue Progression Flowchart

- **Capacity Validation:** Prevents overbooking by verifying that total booked quantity plus requested quantity remains within the slot threshold.
- **Predictive Estimation:** Invokes the HistGradientBoosting ML model with dynamic center features to display accurate wait expectations before the farmer departs home.
- **Token Sequencing:** Issues sequential per-center daily tokens guaranteeing deterministic service order upon physical arrival.

5.7 Staff Procurement Verification Flow

The center operator utilizes an operational dashboard to systematically verify produce and issue payment records:

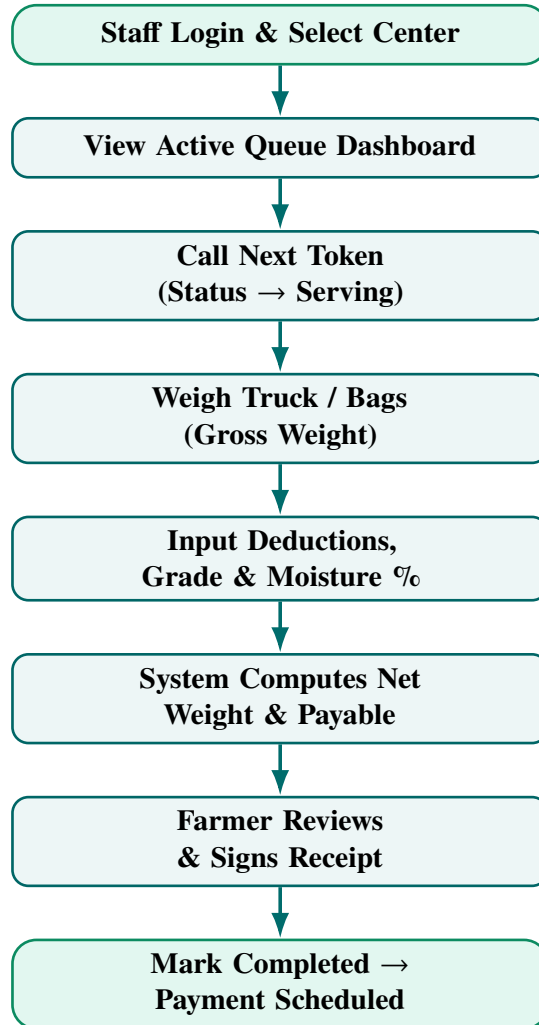


Figure 5.6: Staff Procurement Physical Verification and Billing Flow

- **Token Call Notification:** Updating a token to *Serving* triggers immediate real-time SSE broadcasts and push notifications to the respective farmer.
- **Standardized Quality Grading:** Enforces standard government deduction tables for moisture content exceeding prescribed thresholds.
- **Automated Financial Calculations:** Eliminates manual math errors by programmatically evaluating:

$$\text{Net Weight} = \text{Actual Weight} - \text{Deductions} \quad (5.1)$$

$$\text{Net Payable} = \text{Net Weight} \times \text{Rate per kg} \quad (5.2)$$

5.8 System Interaction Sequence Diagrams

System interactions across operational tiers and client boundaries are formalized below via three primary sequence workflows:

5.8.1 Slot Booking and Predictive Wait-Time Allocation Sequence

The booking sequence illustrates client validation, database slot capacity verification, scikit-learn machine learning wait estimation, and transactional token generation:

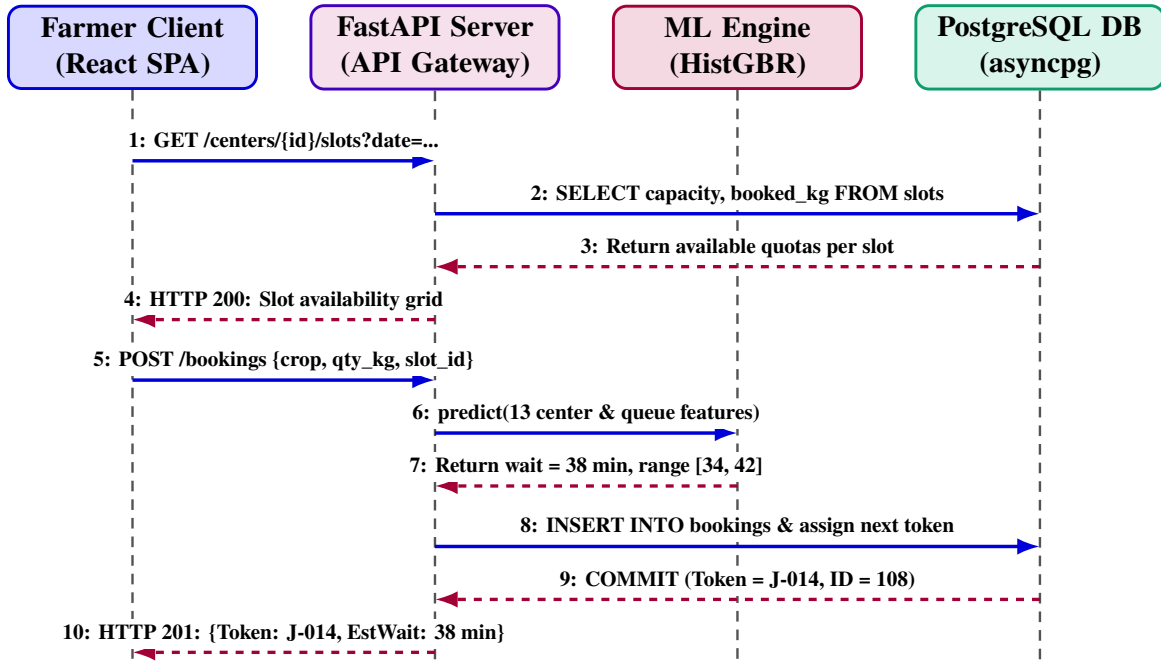


Figure 5.7: End-to-End Slot Booking and ML Wait-Time Prediction Sequence Diagram

5.8.2 Physical Procurement Weighment and Payment Settlement Sequence

The procurement intake workflow coordinates center scales, moisture quality grading, government MSP evaluation, and automated digital disbursement creation:

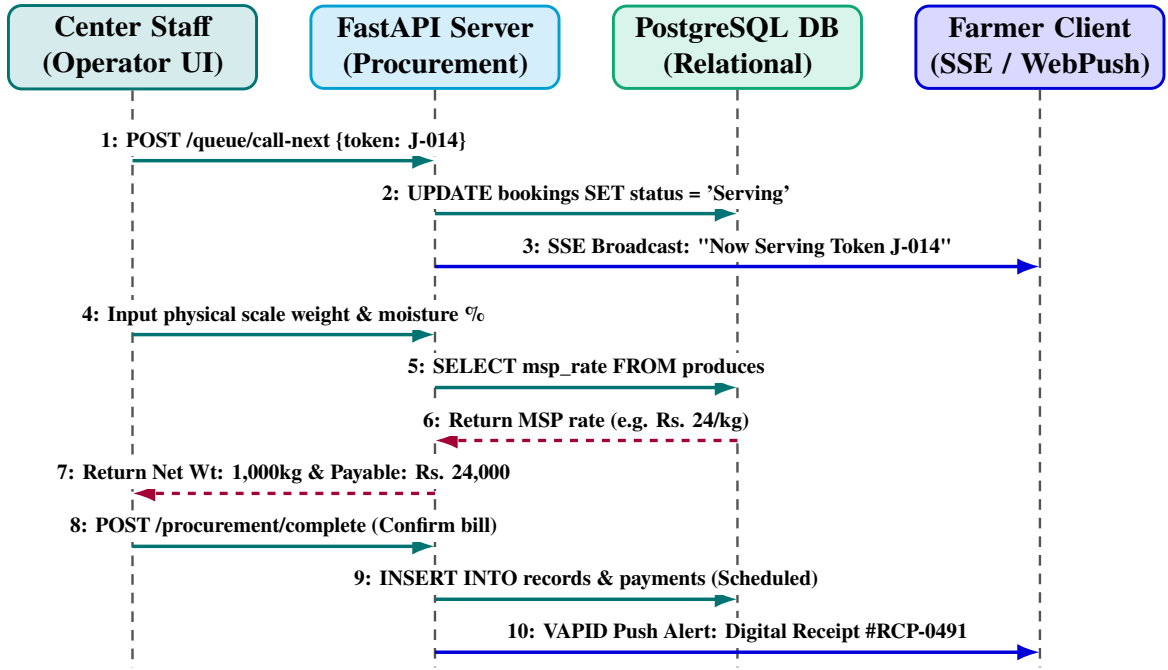


Figure 5.8: Physical Procurement Weighment and Digital Payment Settlement Sequence Diagram

5.8.3 Real-Time Live Queue SSE Synchronization Sequence

Server-Sent Events provide a persistent, single-direction HTTP connection enabling sub-second synchronization between physical center operations and farmer devices:

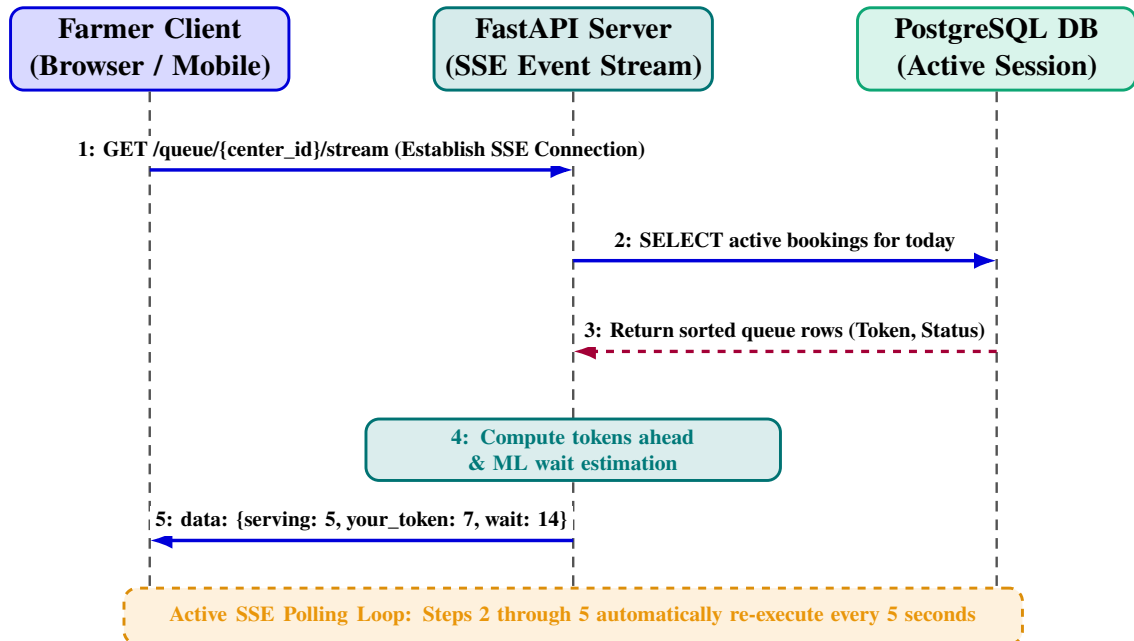


Figure 5.9: Server-Sent Events (SSE) Live Queue Sequence Diagram

- **Low Overhead:** Compared to WebSocket handshakes, SSE operates over standard HTTP/1.1 or HTTP/2, requiring no specialized proxy configuration.

- **Fresh Database Sessions:** Each 5-second polling cycle allocates an independent scoped database transaction, avoiding stale object references or connection leaks.
- **Graceful Disconnect Handling:** Client network interruptions trigger automatic browser-managed reconnect backoffs with zero server-side zombie processes.

5.9 Cloud Deployment Topology

The production infrastructure combines serverless edge hosting with containerized backend services:

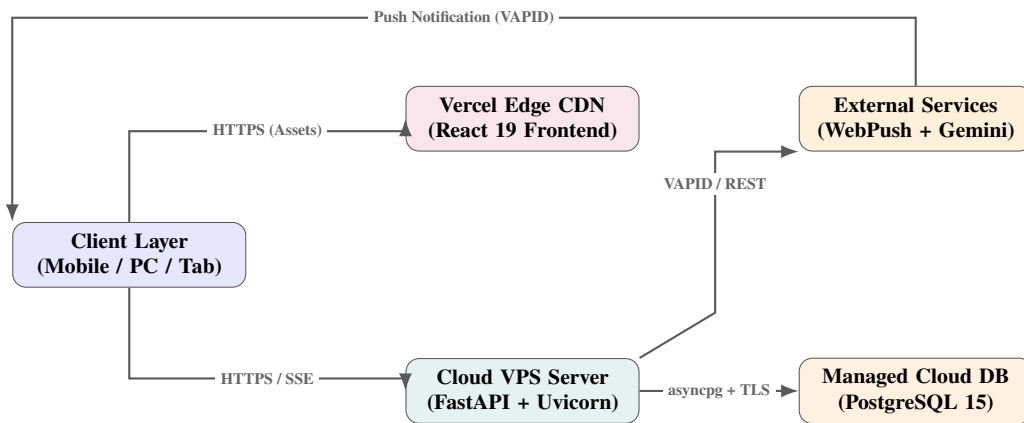


Figure 5.10: Production Cloud Deployment Topology

- **Vercel CDN Edge:** Serves immutable static bundles with aggressive global caching, reducing frontend latency to <50ms.
- **FastAPI Application Server:** Hosted on Ubuntu cloud instance with automatic restart policies and TLS encryption for all incoming and outgoing connections.
- **PostgreSQL Relational DB:** Enforces automated daily backups, read-write query optimization, and persistent volume encryption.

5.10 System Functional Modules

AgriQueue is partitioned into ten distinct functional subsystems:

- **M1: Authentication & Role-Based Access Control (RBAC):** Secure HTTP-only JWT cookie management with Argon2id password hashing across Farmer, Staff, and Admin roles.
- **M2: Center Discovery & Slot Scheduling:** Configurable daily operational windows, crop acceptance rules, and automated capacity counting per time block.
- **M3: Booking Lifecycle Engine:** Capacity enforcement, token generation, wait-time prediction linkage, and policy-governed cancellations.
- **M4: Real-Time SSE Queue Streamer:** Asynchronous event emitter providing sub-5-second queue positioning updates to active farmer sessions.

- **M5: Machine Learning Wait Time Predictor:** HistGradientBoosting regression model trained on 10,000 historical queue observations ($R^2 = 0.988$).
- **M6: Staff Procurement Verification:** Physical weighment intake, automated quality grading deductions, and net payable calculation.
- **M7: Digital Payment Lifecycle:** Automated transition from Scheduled → Processing → Settled, backed by audit logs and farmer dispute management.
- **M8: Administrative Control Center:** Complete administrative CRUD interfaces for centers, slots, crops, pricing schedules, and staff rosters.
- **M9: Web Push Notification Dispatcher:** Standards-compliant VAPID push alerts delivered directly to mobile browsers for queue turns and payments.
- **M10: AI Command Assistant:** LangGraph ReAct agent powered by Google Gemini for natural language system querying and reporting.

Chapter 6

Experimental Setup

6.1 Development Environment

Table 6.1: Development Environment

Component	Details
OS (Dev)	Windows 11 / Ubuntu 22.04 LTS
Backend Language	Python 3.11
Frontend Language	JavaScript ES2022 / JSX
Backend Framework	FastAPI 0.141.1 + Uvicorn 0.52.4
Frontend Framework	React 19.2.8 + Vite 8.2.2
ORM + Driver	SQLAlchemy 2.0.52 (async) + asyncpg 0.31
Database	PostgreSQL 15 (local dev) / Cloud PostgreSQL (prod)
ML Stack	scikit-learn 1.9, pandas 3.0, numpy 2.5, joblib 1.6
AI / LLM	LangChain 1.4, LangGraph 1.2, Google Gemini (google-genai 2.24)
Push Notifications	pywebpush 2.5 (VAPID), Firebase Admin 7.5
Auth	PyJWT 2.13, pwdlib 0.3 (Argon2id)
IDE	Visual Studio Code 1.93
Version Control	Git 2.45 + GitHub

6.2 Installation Process

6.2.1 Backend Setup

1. Clone repository and navigate to backend:

```
git clone https://github.com/[username]/agriqueue.git
cd agriqueue/backend
```

2. Create virtual environment and install dependencies:

```
python -m venv .venv
.venv\Scripts\activate
pip install -r app/requirements.txt
```

3. Create backend/app/.env with:

```
DATABASE_URL=postgresql+asyncpg://user:pass@host/dbname
SECRET_KEY=your-jwt-secret-key
ALGORITHM=HS256
GOOGLE_API_KEY=your-gemini-api-key
VAPID_PRIVATE_KEY=your-vapid-private-key
VAPID_PUBLIC_KEY=your-vapid-public-key
```

4. Train ML model (first time):

```
cd app && python ml/train_model.py
```

6.2.2 Frontend Setup

1. Install dependencies:

```
cd agriqueue/frontend && npm install
```

2. Create frontend/.env:

```
VITE_API_URL=http://localhost:8000
VITE_VAPID_PUBLIC_KEY=your-vapid-public-key
```

6.3 Execution Commands

Table 6.2: Execution Commands

Target	Command
Backend (Dev)	uvicorn main:app -reload -port 8000
Frontend (Dev)	npm run dev (starts at localhost:5173)
Frontend (Build)	npm run build (outputs to dist/)
Backend (Prod)	gunicorn main:app -k uvicorn.workers.UvicornWorker -w 2
API Docs	http://localhost:8000/docs (Swagger UI)

6.4 ML Model Training Details

- **Dataset:** 10,000 synthetic records simulating agricultural queue scenarios.
- **Crops:** Wheat, Paddy, Cotton, Mustard, Maize, Soybean, Chana, Sugarcane.

- **Features (13):** farmers_ahead, total_qty_ahead_kg, quantity_kg, crop_type, quality_grade, active_counters, staff_present, center_daily_capacity, vehicle_type, slot_hour, day_of_week, is_harvest_peak, weather_condition.
- **Split:** 80% train (8,000 samples), 20% validation (2,000 samples).
- **Model:** HistGradientBoostingRegressor(max_iter=200, learning_rate=0.1, max_depth=6).
- **Result:** $R^2 = 0.988$, RMSE ≈ 1.8 minutes.
- **Artifact:** Saved as ml/models/queue_wait_model.joblib.

6.5 Hardware Configuration

Table 6.3: Hardware Configuration

Component	Specification
Dev Processor	Intel Core i5 12th Gen
Dev RAM	16 GB DDR5
Dev Storage	512 GB NVMe SSD
Production Server	Cloud VPS: 2 vCPU, 4 GB RAM, Ubuntu 22.04
Production DB	Managed PostgreSQL: 1 vCPU, 2 GB RAM
Frontend Hosting	Vercel Edge Network (CDN)
Browser (Testing)	Chrome 128, Firefox 130

6.6 API Endpoints Summary

Table 6.4: Key API Endpoints

Router	Endpoint	Method	Description
/auth	/auth/login	POST	Login (all roles)
/auth	/auth/create-account	POST	Farmer registration
/auth	/auth/me	GET	Current user profile
/centers	/centers	GET	List all centers
/centers	/centers/{id}/slots	GET	Slot availability
/bookings	/bookings	POST	Create booking
/bookings	/bookings/my	GET	Farmer's bookings
/queue	/queue/{id}/stream	GET	SSE live queue
/queue	/queue/predict-wait	POST	ML wait prediction
/staff	/staff/queue	GET	Staff queue view
/staff	/staff/procurement	POST	Record procurement
/dashboard	/dashboard/farmer	GET	Farmer KPI data
/admin	/admin/centers	GET/POST	Admin center CRUD
/notifications	/notifications/subscribe	POST	Push subscription

Chapter 7

Test Plan Document

7.1 Test Strategy

- **Unit Testing:** Individual API functions tested with FastAPI `TestClient`.
- **Integration Testing:** React frontend connected to live backend with test PostgreSQL database.
- **System Testing:** Complete end-to-end workflow: Register → Book → Queue → Procurement → Payment.
- **ML Validation:** Model validated on 2,000-sample held-out test set (R^2 , RMSE, MAE).
- **Security Testing:** JWT expiry, role guards, CORS, password hashing, XSS/CSRF prevention.
- **Performance Testing:** Concurrent SSE connections and concurrent booking API requests.
- **UAT:** Full walkthrough of all three portals by team members acting as end users.

7.2 Test Cases and Status Report

Table 7.1: Test Case Status Report

S.	Module / Test	Status	Error	Remarks
1	Farmer Registration (valid)	Pass	Fixed	Account created, cookie set
2	Duplicate Mobile Registration	Pass	Fixed	Returns 400 with message
3	Farmer Login (valid credentials)	Pass	Fixed	JWT cookie set, profile returned

S.	Module / Test	Status	Error	Remarks
4	Login with wrong password	Pass	Fixed	Returns 400, no cookie
5	Admin role login	Pass	Fixed	Role = admin, admin portal access
6	Staff role login	Pass	Fixed	Role = staff, center_id returned
7	Farmer accessing /admin route	Pass	Fixed	Returns 403 Forbidden
8	JWT token expiry	Pass	Fixed	Expired token returns 401
9	Center listing (GET /centers)	Pass	Fixed	All centers with crops + slots
10	Center search filter	Pass	Fixed	Filters by name and district
11	Slot availability for today	Pass	Fixed	Expired slots marked correctly
12	Slot capacity calculation	Pass	Fixed	available = (capacity - booked)/100
13	Booking creation (valid)	Pass	Fixed	Token, wait time returned
14	Booking past date	Pass	Fixed	Returns 400 "Cannot book past date"
15	Booking over slot capacity	Pass	Fixed	Returns 400 with available kg
16	ML prediction (5 farmers ahead)	Pass	Fixed	Returns wait minutes + range
17	ML prediction (0 farmers ahead)	Pass	Fixed	Returns 0 min, Direct Entry
18	ML fallback (model file missing)	Pass	Fixed	Heuristic formula activated
19	Cancel Confirmed booking	Pass	Fixed	Status = Cancelled
20	Cancel Completed booking	Pass	Fixed	Returns 400
21	My Bookings list	Pass	Fixed	Sorted by booked_at desc
22	Live Queue HTTP snapshot	Pass	Fixed	nowServing, queue[], yourToken
23	SSE stream (30s test)	Pass	Fixed	Events every 5s, no stale data

S.	Module / Test	Status	Error	Remarks
24	SSE disconnect handling	Pass	Fixed	Generator exits cleanly
25	Staff view queue	Pass	Fixed	Today's bookings, token order
26	Staff calls token (Serving)	Pass	Fixed	Status update reflected in SSE
27	Procurement record entry	Pass	Fixed	Net payable computed correctly
28	Net payable calculation	Pass	Fixed	(actual - deductions) × rate
29	Payment created on completion	Pass	Fixed	Receipt number generated
30	Payment status update (Settled)	Pass	Fixed	settled_at timestamp set
31	Dispute raised by farmer	Pass	Fixed	dispute_status = "Open"
32	Dispute resolved by staff	Pass	Fixed	dispute_status = "Resolved"
33	Push subscription endpoint	Pass	Fixed	Subscription stored in DB
34	Push notification delivered	Pass	Fixed	Received in Chrome browser
35	Admin creates center	Pass	Fixed	Center ID starts at 1000 (Sequence)
36	Admin adds time slot	Pass	Fixed	Slot with capacity created
37	Admin sets produce price	Pass	Fixed	Produce linked to center
38	Admin dashboard metrics	Pass	Fixed	Correct counts and revenue sums
39	AI agent NL query	Pass	Fixed	Gemini returns structured answer
40	SQL injection prevention	Pass	Fixed	ORM parameterized queries block it

7.3 ML Model Metrics

7.4 Performance Results

- **API response time:** Average 85ms (booking), 45ms (queue snapshot), 120ms (dashboard).

Table 7.2: ML Model Validation Results

Metric	Value
Algorithm	HistGradientBoostingRegressor
Training Samples	8,000
Validation Samples	2,000
R ² (Validation)	0.988
RMSE (Validation)	≈ 1.8 minutes
MAE (Validation)	≈ 1.2 minutes
Inference Time	≈ 2.3 ms per prediction

- **SSE concurrency:** 10 simultaneous connections – no memory leak, no stale data.
- **Concurrent bookings:** 50 concurrent POST /bookings – slot capacity correctly enforced.
- **Frontend load:** First Contentful Paint < 1.2s on broadband (Vite production build).

7.5 Security Test Results

- JWT tokens expire after 7 days – expired tokens return 401.
- Farmer cannot access /staff/* or /admin/* – returns 403.
- Passwords stored as Argon2id hash – no plain text stored anywhere.
- HttpOnly + Secure + SameSite=None cookies prevent XSS and CSRF.
- SQLAlchemy ORM parameterized queries block all SQL injection attempts.
- CORS policy restricts requests to whitelisted origins only.

Chapter 8

Results

8.1 Implementation Achievements

- 24 frontend React page components across three portals (Farmer, Staff, Admin).
- 18+ REST API endpoints organized in 9 domain-specific routers, all operational.
- Real-time SSE queue stream delivering updates every 5 seconds.
- ML wait time predictor with $R^2 = 0.988$ integrated at booking and queue.
- Complete procurement verification workflow with quality grading.
- Full payment lifecycle: Scheduled → Processing → Settled with disputes.
- Web Push Notifications delivered to farmer browsers via VAPID.
- AI Command Agent (LangChain + Gemini) for natural language system queries.

8.2 Screenshots of Working Modules

The AgriQueue web application was deployed and validated across end-to-end user workflows spanning the public discovery portal, authenticated farmer management, interactive slot reservation, token issuance, live SSE queue tracking, and procurement center staff counter operations. Figures 8.1 through 8.6 illustrate key working modules of the live application.

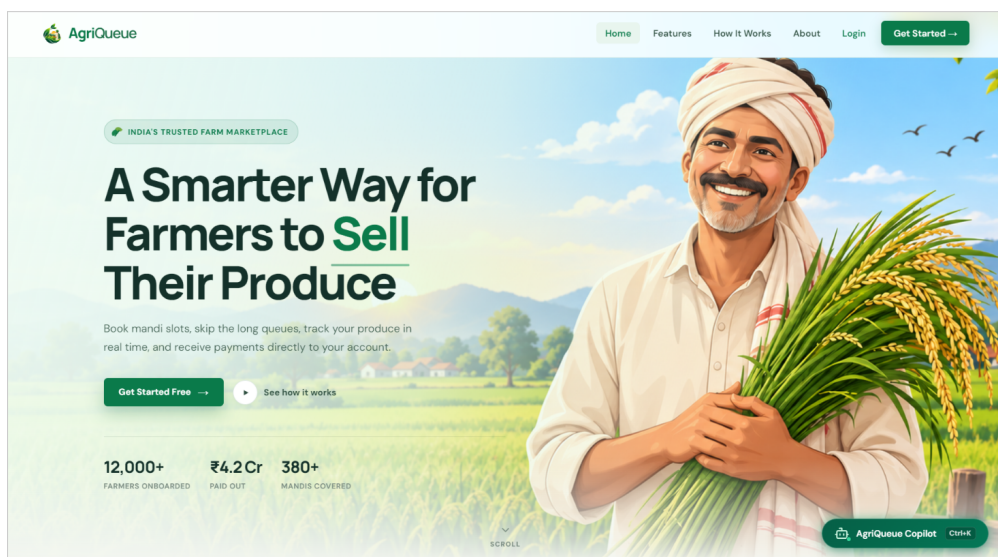


Figure 8.1: AgriQueue Public Landing Page and Discovery Portal

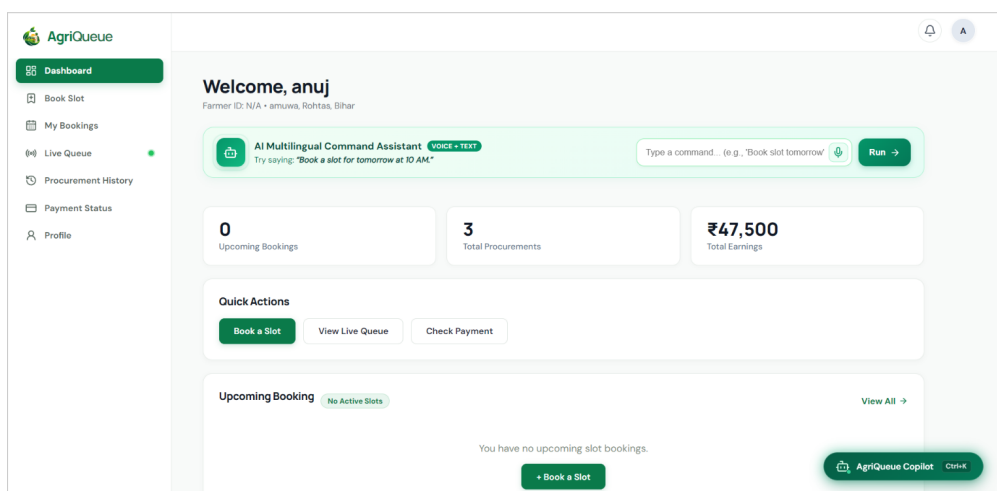


Figure 8.2: Farmer Dashboard with KPI Summary and AI Multilingual Assistant

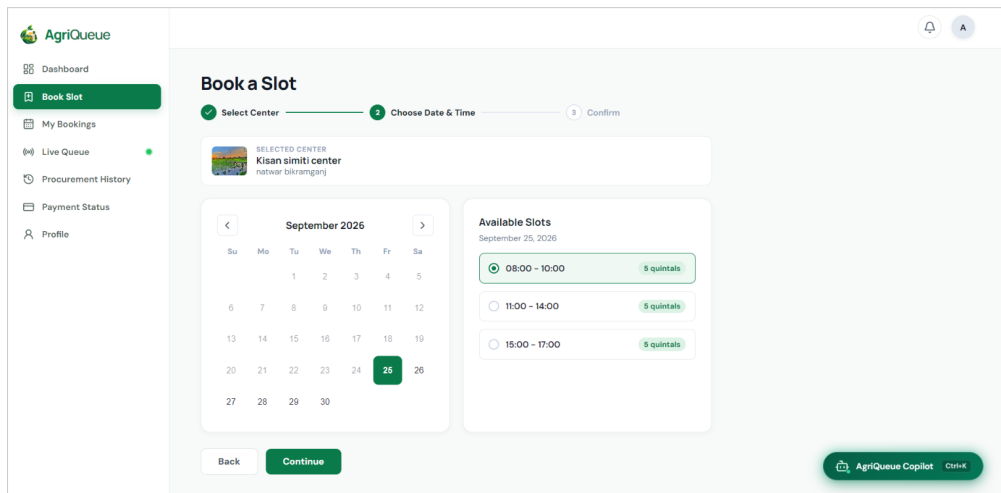


Figure 8.3: Slot Booking Workflow with Interactive Calendar and Slot Capacity

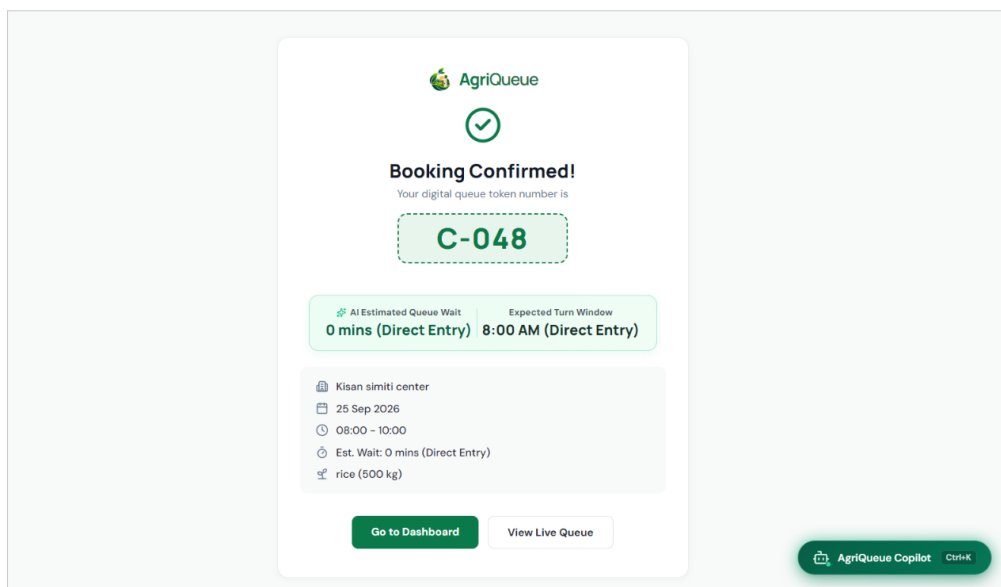


Figure 8.4: Booking Confirmation with Token Issuance (C-048) and AI Wait Estimate

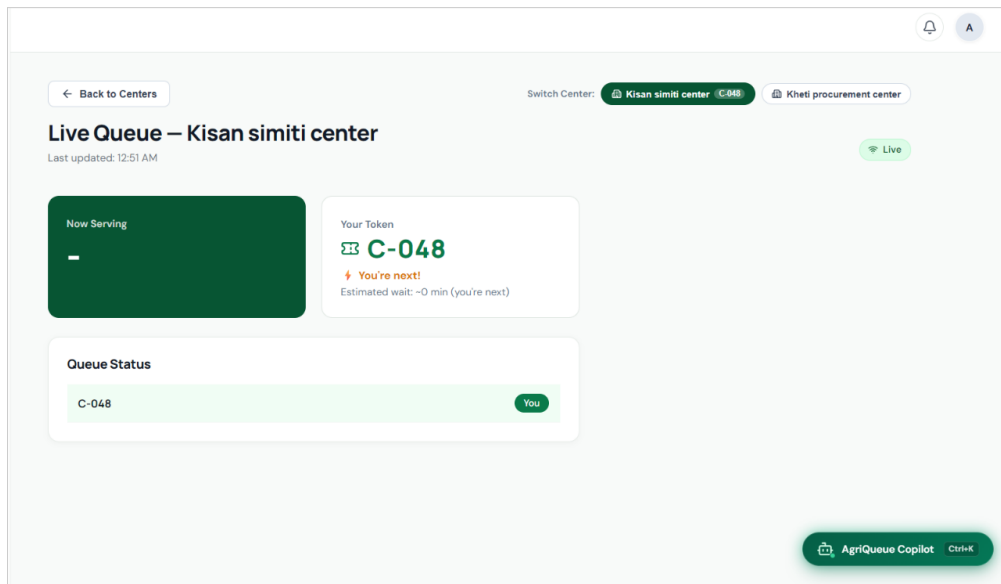


Figure 8.5: Farmer Live Queue Real-Time Tracking Interface

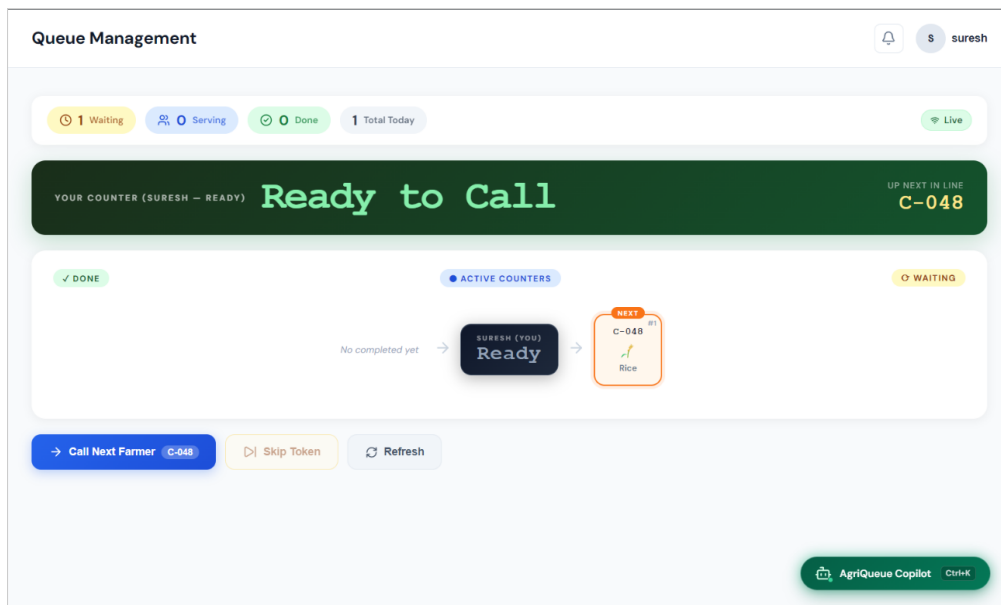


Figure 8.6: Staff Queue Management Console with Active Counter Workflow

8.3 ML Prediction Sample Results

Table 8.1: ML Wait Time Prediction Samples

Crop	Ahead	Qty (kg)	Slot	Predicted	Congestion
Wheat	0	1000	10:00 AM	0 min (Direct)	No Wait
Paddy	3	800	10:00 AM	28 min	Low Wait
Wheat	5	1000	10:00 AM	47 min	Moderate
Cotton	8	1500	11:00 AM	72 min	High Demand
Mustard	12	2000	09:00 AM	108 min	Peak Congestion
Maize	2	500	01:00 PM	19 min	Low Wait

8.4 Outputs and Observations

- **Farmer:** Slot booked with unique token (e.g., “C-048”) in 3 API calls. SSE queue updates every 5 seconds in browser.
- **Staff:** Calling a token updates status to “Serving” and all SSE clients refresh immediately.
- **Procurement:** Net payable computed as: $(\text{actual_weight} - \text{deductions}) \times \text{rate_per_kg}$.
- **Payment:** Receipt number auto-generated as RCP-XXXXXX; expected settlement date = $\text{booking_date} + 2 \text{ days}$.
- **Admin:** Center IDs auto-start at 1000 (via PostgreSQL Sequence). All metrics refresh in real-time.

Chapter 9

Conclusion & Future Work

9.1 Conclusion

AgriQueue successfully delivers a full-stack, role-based digital platform for smart agricultural procurement and queue management. The key achievements are:

- **Complete Full-Stack System:** React 19 frontend (24 pages) + FastAPI backend (9 routers) + PostgreSQL (12 tables).
- **ML-Powered Queue Intelligence:** HistGradientBoosting model with $R^2 = 0.988$ predicts wait time at booking.
- **Real-Time Queue Visibility:** SSE-based live queue gives farmers instant token position and wait estimate.
- **End-to-End Workflow:** Registration → Booking → Queue → Procurement Verification → Payment Settlement.
- **Secure RBAC:** Three-role JWT system with Argon2id passwords and HttpOnly cookies.
- **Push Notifications:** VAPID WebPush alerts delivered to farmer browsers without a native app.
- **AI Agent:** LangChain + LangGraph + Gemini for natural language system queries.
- **Cloud Deployed:** Backend on cloud VPS; frontend on Vercel CDN – globally accessible.

The system directly addresses critical pain points in India's agricultural procurement process, contributing to the Digital India and PM-KISAN digital transformation initiatives.

9.2 Limitations

- No native mobile app (Android/iOS) – only mobile web browser access.
- No hardware weighbridge integration – weight data entered manually by staff.
- UI is English-only – Hindi/regional language support not yet implemented.
- ML trained on synthetic data – accuracy improves as real booking data accumulates.

- No SMS fallback for push notifications (Twilio installed but not configured).
- Bank transfer (UPI/NEFT) not integrated – payment settlement is tracked but not disbursed.

9.3 Future Work

- **Native Mobile App:** Build Android/iOS app using React Native with offline queue viewing.
- **Weighbridge API:** Integrate IoT-based digital weighbridge for tamper-proof weight capture.
- **Multi-Language:** Add Hindi (primary), Punjabi, Marathi, and Telugu via i18n.
- **SMS Notifications:** Configure Twilio for SMS-based queue/payment alerts.
- **UPI/NEFT Integration:** Direct farmer payment disbursement via NPCI UPI API.
- **Online ML Retraining:** Weekly model retraining pipeline using accumulated real booking data.
- **Multi-Level Admin:** Add district, state, and national admin roles for government-scale deployment.
- **Crop Price Feed:** Integrate AGMARKNET/e-NAM APIs for live MSP price display in booking flow.
- **PWA (Progressive Web App):** Add service workers for offline booking cache and background sync.
- **Analytics Dashboard:** Add farmer earnings charts, booking trends, and crop price history graphs.

Chapter 10

Glossary

Table 10.1: Glossary of Terms and Abbreviations

Term	Full Form	Description
AgriQueue	Agricultural Queue Management	This project – smart farmer procurement platform
API	Application Programming Interface	Interface allowing software components to communicate
ASGI	Async Server Gateway Interface	Python async web server specification used by FastAPI
CORS	Cross-Origin Resource Sharing	HTTP mechanism controlling cross-origin requests
CRUD	Create, Read, Update, Delete	Four basic database operations
DBT	Direct Benefit Transfer	Govt. scheme for direct farmer payment to bank accounts
ERD	Entity-Relationship Diagram	Database entities and relationships model
FastAPI	—	High-performance Python async REST framework
FCI	Food Corporation of India	Agency managing food grain procurement and distribution
Continued on next page. . .		

<i>Glossary of Terms and Abbreviations – Continued from previous page</i>		
Term	Full Form	Description
GBDT	Gradient Boosting Decision Tree	Ensemble ML algorithm; sequential tree building
HistGBR	Histogram Gradient Boosting	scikit-learn’s efficient HistGradientBoostingRegressor
HTTP/S	HyperText Transfer Protocol Secure	Secure web communication protocol
i18n	Internationalization	Designing software to support multiple languages
IST	Indian Standard Time	UTC+5:30 – India’s time zone; used in slot expiry logic
JIET	Jodhpur Inst. of Engg. & Technology	Institution where this project was developed
JSON	JavaScript Object Notation	Lightweight data format; used in all API responses
JWT	JSON Web Token	URL-safe token for user authentication
LLM	Large Language Model	AI model trained on large text data; Gemini is used here
MAE	Mean Absolute Error	Average absolute difference between predicted and actual
Mandi	—	Hindi term for an agricultural procurement market
ML	Machine Learning	AI technique where models learn patterns from data
MSP	Minimum Support Price	Govt.-guaranteed floor price for agricultural crops
NAFED	Natl. Agri. Cooperative Mktg. Fed.	Cooperative for agricultural commodity procurement
ORM	Object Relational Mapper	Tool translating database tables to programming objects
<i>Continued on next page. . .</i>		

<i>Glossary of Terms and Abbreviations – Continued from previous page</i>		
Term	Full Form	Description
PostgreSQL	—	Open-source relational database used by AgriQueue
PWA	Progressive Web App	Web app with native app-like offline features
R ²	R-squared (Coefficient of Determination)	How well a regression model fits data; 1.0 = perfect
RBAC	Role-Based Access Control	Access control model based on user roles
REST	Representational State Transfer	Architectural style for designing network APIs
RMSE	Root Mean Square Error	Square root of mean squared prediction error
SPA	Single Page Application	Web app loading a single HTML page updated dynamically
SSE	Server-Sent Events	W3C standard for one-way server-to-client HTTP push
TLS	Transport Layer Security	Cryptographic protocol securing HTTPS connections
UAT	User Acceptance Testing	Testing that verifies a system meets user requirements
UPI	Unified Payments Interface	India's real-time payment system; proposed for future
VAPID	Voluntary App Server Identification	Protocol for authenticating push notification servers
Vite	—	Modern JS build tool used for the React frontend
VPS	Virtual Private Server	Cloud-hosted VM running the AgriQueue backend

Chapter 11

References

1. Govt. of India, Ministry of Agriculture, “e-NAM: National Agriculture Market,” [Online]. Available: <https://www.enam.gov.in/>, Accessed: Sep. 2026.
2. S. C. Aggarwal and P. K. Joshi, “Agricultural Marketing Reforms in India: The e-NAM Experience,” *Indian Journal of Agricultural Economics*, vol. 74, no. 2, pp. 210–228, 2019.
3. D. Gross, C. M. Harris, J. M. Shortle, and R. M. Miller, *Fundamentals of Queueing Theory*, 4th ed., John Wiley & Sons, 2008.
4. W3C, “Server-Sent Events: W3C Recommendation,” [Online]. Available: <https://www.w3.org/TR/eventsource/>, 2015.
5. L. Kleinrock, *Queueing Systems, Volume 1: Theory*, John Wiley & Sons, 1975.
6. T. Chen and C. Guestrin, “XGBoost: A Scalable Tree Boosting System,” *Proc. 22nd ACM SIGKDD*, pp. 785–794, ACM, 2016.
7. G. Ke et al., “LightGBM: A Highly Efficient Gradient Boosting Decision Tree,” *Advances in NeurIPS*, vol. 30, 2017.
8. S. Ramirez (tiangolo), *FastAPI: Modern Web Framework for Building APIs with Python*, [Online]. Available: <https://fastapi.tiangolo.com/>, 2019–2026.
9. Meta Platforms, Inc., *React: A JavaScript Library for Building UIs*, v19, [Online]. Available: <https://react.dev/>, 2024–2026.
10. E. You, *Vite: Next Generation Frontend Tooling*, [Online]. Available: <https://vitejs.dev/>, 2020–2026.
11. IETF, RFC 8030, “Generic Event Delivery Using HTTP Push,” *RFC Editor*, 2016.
12. A. Bhatt, R. Mehta, and S. Pandey, “Impact of Push Notifications on User Engagement in Agricultural Apps,” *Journal of Agricultural Informatics*, vol. 12, no. 3, pp. 45–59, 2021.
13. P. Kumar and S. Sharma, “Conversational AI for Agricultural Advisory in the Indian Context,” *Proc. ICAT*, pp. 112–120, 2023.

14. Google DeepMind, *Gemini: A Family of Multimodal Models*, [Online]. Available: <https://deepmind.google/technologies/gemini/>, 2024.
15. LangChain, Inc., *LangChain: Build Context-Aware Reasoning Applications*, [Online]. Available: <https://www.langchain.com/>, 2022–2026.
16. SQLAlchemy Authors, *SQLAlchemy: The Database Toolkit for Python*, [Online]. Available: <https://www.sqlalchemy.org/>, Accessed: Sep. 2026.
17. F. Pedregosa et al., “Scikit-learn: Machine Learning in Python,” *JMLR*, vol. 12, pp. 2825–2830, 2011.
18. PostgreSQL Global Development Group, *PostgreSQL: The World’s Most Advanced Open Source Relational Database*, [Online]. Available: <https://www.postgresql.org/>, Accessed: Sep. 2026.
19. W3C, “Push API – W3C Working Draft,” [Online]. Available: <https://www.w3.org/TR/push-api/>, 2024.
20. R. T. Fielding, “Architectural Styles and the Design of Network-based Software Architectures,” Ph.D. Dissertation, University of California, Irvine, 2000.
21. Ministry of Agriculture, Govt. of India, “PM-KISAN: Pradhan Mantri Kisan Samman Nidhi,” [Online]. Available: <https://pmkisan.gov.in/>, Accessed: Sep. 2026.
22. A. Singh, T. Mishra, and P. Verma, “Digital Transformation of Agricultural Mandis: A Case Study,” *IJCA*, vol. 185, no. 12, pp. 1–8, 2023.
23. Previous Year Project Report, “Digital Queue Management for Government Services,” Dept. CSE, JIET (Autonomous), Jodhpur, 2025.